

Сопроводительные материалы к ДЗ №2

Промпты, команды, приёмы, ссылки · Занятие 2

2026-08-22

Как этим пользоваться

Здесь собрано всё, что понадобится для домашнего задания: с чего начать, как спавнить агентов, как собрать workflow, как работать с worktrees, как сделать красивый PDF и где брать инструменты.

Промпты можно копировать как есть — они написаны так, чтобы работать и в Claude Code, и в Codex.

Главный приём занятия, который стоит применять ко всему остальному: всё, чего вы не знаете и в чём сомневаетесь, прогоняйте через самого агента. В большинстве случаев это закрывает вопрос быстрее, чем поиск в интернете.

Обязательных пунктов в задании три: работающий результат, агенты, workflow. Остальные разделы здесь — для тех, кто берёт необязательные пункты в плюс.

1. С чего начать

Superpowers — агентная обвязка из коробки

Если не хочется собирать оркестрацию с нуля — начните отсюда. Superpowers ставится одной командой и сразу даёт небольшой рабочий набор: скиллы, хуки и готовые процедуры.

Главное, ради чего он нужен в этом задании: **его хуки не дают проскочить этап проектирования**. Если вы забыли написать спецификацию или продумать задачу — он остановит и задаст вопросы. Чем раньше поправлены мысли модели, тем меньше переделок в конце.

Он же предложит оформить ваш рабочий процесс как сохранённый workflow — это прямо закрывает обязательный пункт 3.

- Репозиторий: <https://github.com/obra/superpowers> (276 тыс. звёзд на 22.08.2026)

Расскажи про репозиторий <https://github.com/obra/superpowers>: что он делает, что ставит на мою машину, какие хуки добавляет и что изменится в моей работе после установки. Отдельно — как его установить и как потом удалить, если не понравится.

Голосовой ввод — не обязателен, но сильно экономит время

Промпты в этом задании длинные. Набирать их руками — три-пять минут на каждый; надиктовать — двадцать секунд. На занятии это было показано вживую, и разница в длине промптов была заметна сразу: голосом ставится больше задач, описывается целый флоу вместо односложных «продолжай».

Typeless — то, что использовалось в эфире. Одна кнопка, говоришь — текст появляется там, где курсор. Что делает сверх распознавания:

- убирает слова-паразиты, задумчивость, кривые фразы;
- если одну мысль сказали дважды — записывает один раз;
- термины вроде Docker, Nginx, Swift, Playwright пишет правильно внутри русского текста;
- умеет списки по пунктам и перевод на другой язык.

Сайт: <https://www.typeless.com> — macOS, Windows, iOS, Android.

Альтернативы:

- **Wispr Flow** — <https://wisprflow.ai>, тот же класс инструмента, облачная обработка;
- **бесплатные локальные решения на Whisper** — ничего не уходит в облако, работает на Apple Silicon, занимает пару гигабайт памяти. Например, OpenWhisper (<https://github.com/OpenWhisper/openwhisper>, кроссплатформенный, локальные и облачные модели) или любой из десятка аналогов по запросу `whisper dictation macOS` на GitHub;
- локальная модель через **Ollama** (<https://ollama.com>), если хочется собрать самому.

Попробуйте хотя бы раз. Это одна из тех вещей, которые кажутся мелочью, пока не попробуешь.

2. Агенты: как спавнить и как замерить

Где это работает по умолчанию

ИНСТРУМЕНТ	СУБАГЕНТЫ	КАК ЗВАТЬ
Claude Code	есть из коробки	словами: «сделай тремя параллельными агентами»
Codex	есть	так же, словами
Китайские модели (GLM, Kimi, MiniMax и другие)	зависит от CLI	проверять индивидуально перед выбором

Документация: субагенты в Claude Code — <https://code.claude.com/docs/en/sub-agents>. Каждый субагент работает в своём контекстном окне со своим системным промптом и возвращает в главную сессию только результат.

Проверить, умеет ли ваш инструмент спавнить агентов:

Умеешь ли ты запускать субагентов параллельно? Если да — покажи, как ты это делаешь, запусти двух пробных на простой задаче и покажи список запущенных. Если нет — скажи прямо, что не умеешь.

Промпт на параллельный прогон

Раздели эту задачу на три независимые части и запусти по субагенту на каждую, параллельно. Каждому дай свою зону и запрети трогать чужую. Когда все закончат — собери результаты в один документ и скажи, сколько заняло по времени и сколько контекста ты потратил.

Задача: <ваша задача>

Тот же прогон последовательно — если хотите сравнить

Ту же самую задачу сделай сам, последовательно, без субагентов. В конце скажи, сколько это заняло и сколько контекста ушло.

	ПОСЛЕДОВАТЕЛЬНО	ТРИ СУБАГЕНТА
Время		
Контекст главной сессии		
Качество результата		

Не удивляйтесь, если параллельно окажется медленнее. На занятии живой замер дал именно это: каждый субагент тянет свой системный промпт и стартовые проходы, и на маленькой задаче это дороже, чем сделать всё подряд. Экономится не время, а контекст: три субагента оставили в главной сессии 128 тысяч токенов против почти миллиона при последовательной работе. Выигрыш по времени

появляется там, где частей много и они по-настоящему независимы — например, пять параллельных исследований по пяти конкурентам.

Агент-ломатель (для бэкенда и фронтенда)

Отдельная сессия, отдельный агент. Не тот, что писал код: автор не бывает независимым проверяющим.

Ты — независимый проверяющий. Твоя задача не подтвердить, что сервис работает, а сломать его. Веди себя как человек, который очень хочет, чтобы он упал.

Работай итерациями, минимум 10 попыток. На каждой:

- 1) придумай сценарий, которого автор не предусмотрел;
- 2) выполни его;
- 3) запиши, что произошло.

Что пробовать: кривые и пустые данные, границы диапазонов, повторные отправки, одновременные запросы, неверный порядок вызовов, очень длинные строки, отрицательные и дробные суммы, чужие идентификаторы.

В конце — список найденного, отсортированный по опасности.

Если сломать не удалось — так и напиши, но перечисли, что именно проверил.

3. Workflow: как собрать повторяемую процедуру

Это обязательный пункт. Workflow — записанная последовательность шагов, которую вы будете запускать повторно. Не «хороший длинный промпт на один раз», а сохранённая процедура с предсказуемым результатом.

Формат — любой

ФОРМАТ	КОГДА УДОБЕН
Скилл	процедура нужна во многих проектах; агент сам подхватывает по описанию
Хук	действие должно происходить автоматически до или после чего-то
Workflow-файл	многошаговый сценарий с ветвлениями и проверками
Сохранённый промпт	самый простой вариант: текст в файле, который вы копируете каждый раз

Доказывать повторяемость не нужно. Достаточно приложить файл или текст.

Что взять за основу

Схема ночного прогона с занятия — можно упрощать сколько угодно:

спецификация

→ два независимых ревьюера, по два раунда каждый

→ план

→ проверка плана независимым агентом (два раза)

→ реализация по плану

→ тесты

→ проверка реализации агентами

→ финальная проверка всей работы отдельным независимым агентом

→ отчёт отдельным файлом

Пример попроще, для поддержки: «взять новые обращения из папки → классифицировать → для каждого класса сформировать черновик ответа по плейбуку → сложить в файл на проверку». Пример для QA: «взять изменения с последнего прогона → собрать список затронутых сценариев → прогнать → отчёт». Пример для маркетинга: «взять пять источников → вытащить цифры за неделю → обновить таблицу → написать сводку».

Промпт на создание

Собери workflow, который я смогу запускать повторно.

Название: <ночной прогон / сборка отчёта / разбор обращений>

Что он делает: <опишите словами>

Шаги:

- 1) ...
- 2) ...
- 3) ...

Требования:

- каждый шаг оставляет файл, по которому видно, что он выполнен;
- проверку выполняет отдельный агент, не тот, который делал работу;
- в конце – отчёт: что сделано, что не получилось, что осталось.

Сохрани его в том формате, который удобнее для тебя и для меня (скилл, хук, workflow-файл или просто промпт в файле), и покажи, как именно вызывать.

Если стоит Superpowers — он предложит это сам в конце сессии.

4. Worktrees: три ветки одновременно (необязательно, в плюс)

Зачем

Три субагента в одной папке — это один человек, разделённый на три: они спотыкаются друг о друга, потому что файлы общие. Три агента в трёх worktrees — это **три независимых разработчика**: у каждого своя ветка, своя история, свои коммиты, и в конце — merge.

Документация git: <https://git-scm.com/docs/git-worktree>. В Claude Code субагенту можно выдать собственный worktree одной строкой `isolation: worktree` — см. <https://code.claude.com/docs/en/sub-agents>.

Команды

```
# из корня репозитория
git worktree add ../work-feature-a -b feature-a
git worktree add ../work-feature-b -b feature-b
git worktree add ../work-feature-c -b feature-c

git worktree list          # посмотреть, что создано
```

Дальше в каждой папке запускается свой агент со своей задачей.

Слияние:

```
git merge feature-a
git merge feature-b
git merge feature-c
```

Уборка — обязательно:

```
git worktree remove ../work-feature-a
git worktree prune
```

Про диск. Worktree разворачивает полную рабочую копию файлов (`.git` при этом общий). На большом проекте с зависимостями три ветки — три `node_modules`. В эфире прозвучал случай, когда накопилось 250 ГБ и работа встала. Чистите за собой сразу.

Промпт, если не хочется руками

Мы работаем через `git worktrees`. Если `git`-репозитория ещё нет — инициализируй.
Создай три `worktree` с отдельными ветками и запусти в каждом по агенту:

- 1) <задача 1>
- 2) <задача 2>
- 3) <задача 3>

Каждый агент работает только в своей папке. Когда все закончат — покажи, что получилось в каждой ветке, и предложи порядок слияния.
Мержить буду я, сам не мержи.

Для тех, кто не пишет код

Работает точно так же: репозиторий — это папка с документами. Три раздела исследования, три сценария плейбука, три области проверок — по ветке на каждый. Смысл тот же: увидеть, как три параллельные работы сходятся в одну и что при этом конфликтует.

5. Изоляция (необязательно, желательно попробовать)

Три уровня

УРОВЕНЬ	ЧТО ДАЁТ	КОГДА ХВАТАЕТ
Встроенная песочница Claude Code	ограничивает Bash и его дочерние процессы	обычная работа в своём проекте
Docker или чистая VM	ограничивает всё окружение	сомнительный код, чужой репозиторий, работа без присмотра
Отдельная физическая машина	полностью	если совсем страшно

Встроенная песочница включается командой `/sandbox` в сессии. На macOS ничего ставить не нужно — используется системный механизм Seatbelt; на Linux — bubblewrap. Два режима: **auto-allow** (команды в песочнице выполняются без вопросов) и **regular permissions** (обычные подтверждения сохраняются). По умолчанию команда внутри может писать только в рабочую папку и временную папку сессии, а каждый новый сетевой домен требует подтверждения.

Документация: <https://code.claude.com/docs/en/sandboxing>. Уровень выше — контейнеры и VM: <https://code.claude.com/docs/en/sandbox-environments>.

Главное предупреждение

Встроенная песочница накрывает терминал, но не накрывает MCP и остальную обвязку.

Документация говорит об этом прямо: песочница «применяется только к Bash-командам и их дочерним процессам». Если у вас подключены MCP-серверы, скрипты через UI или AppleScript — они продолжают дотягиваться туда, куда терминалу уже нельзя.

Мы сейчас в песочнице. Проверь границу двумя способами:

- 1) попробуй записать файл за пределами рабочей папки через терминал;
- 2) попробуй сделать то же самое через любой подключённый MCP-сервер.

Покажи вывод обеих попыток. Не обходи ограничение, если оно сработало, — мне нужен именно факт отказа.

Permissions: уточнение к тому, что прозвучало в эфире

На занятии было сказано, что в режиме `bypassPermissions` правила `permissions` не работают. Это верно для `allow` и `ask` — подтверждения отключаются. Но **deny -правила действуют во всех режимах**, включая `bypass`: документация формулирует это как «explicit deny rules still apply». Порядок проверки всегда один: сначала `deny`, потом `ask`, потом `allow`.

Что при этом остаётся верным и было показано вживую: `deny` — это шаблон на строку команды. Запрет на `cat secrets.txt` не остановит `python -c "print(open('secrets.txt').read())"`. Механизм, который не обойти сменой команды, — это `PreToolUse` -хук или песочница.

Секреты

Что прозвучало в эфире прямым текстом: агент **в конце концов получит доступ** к вашим ключам — в какой-то момент вы расслабитесь, он прочитает файл, и ключ окажется в контексте.

Два рабочих способа:

1. **Права на уровне ОС.** Запретить чтение файла с ключами всем, кроме sudo. Тогда агент не получит доступ, что бы ни случилось.
2. **Плейсхолдеры.** В рабочем файле — заглушка, реальное значение подставляется отдельно.

Перед сдачей — проверить репозиторий на секреты: `.env`, `settings.json`, история команд, скриншоты терминала.

6. Недоверенный текст (необязательно, на своё усмотрение)

Куда прячут инструкции

- белым шрифтом по белому фону на странице;
- в HTML-тегах и комментариях;
- в метаданных файла;
- в поле, куда пишет посторонний человек: назначение платежа, комментарий к переводу, текст обращения в поддержку;
- в описании инструмента или скилла, который вы скачали.

Как проверять

Одну и ту же задачу — на слабой и на сильной модели.

СПОСОБ ВНЕДРЕНИЯ

СЛАБАЯ МОДЕЛЬ

СИЛЬНАЯ МОДЕЛЬ

комментарий в файле

выполнила / промолчала / назвала

белый текст

метаданные

поле «назначение платежа»

«служебная строка»

Третий вариант ответа — самый важный. На занятии слабая модель (Haiku) молча не выполнила инъекцию, а сильная назвала все четыре штуки, включая спрятанную. «Молча не сделала» и «сказала вслух» — разные уровни защиты, и полагаться можно только на второй.

7. Инструменты: как искать, проверять и ставить (необязательно)

Хорошая модель сама закрывает базовый context management, а оптимизатор лишь немного сокращает расход токенов. Это раздел для тех, кому интересно.

Как искать

Способ с занятия: сформулировать, что нужно, и добавить к запросу слово GitHub. Почти всё живёт там.

- **оптимизатор контекста** — искать по `context optimizer`, `context management claude code`. Показанные в эфире: `context-mode` — <https://github.com/mksglu/context-mode> (20 тыс. звёзд) и `rtk` — <https://github.com/rtk-ai/rtk> (77 тыс.);
- **индекс кодовой базы** — `GitNexus` — <https://github.com/abhigyanpatwari/GitNexus> (46 тыс.), `Graphify` — <https://github.com/safishamsi/graphify> или **любой аналог**.

Цифры — на 22.08.2026. В эфире они назывались по памяти и местами с ошибкой; верить стоит странице репозитория, а не цифре из ролика или презентации.

Как проверять до установки

Два отдельных запроса, именно в таком порядке:

Вот репозиторий: <ссылка>. Расскажи, что это, зачем нужно, чем отличается от аналогов. Ничего не устанавливай.

Теперь проверь этот репозиторий с точки зрения безопасности: что он ставит на машину, какие хуки добавляет, куда ходит по сети, что читает, есть ли подозрительные места в коде и в описаниях инструментов. Дай вердикт: ставить или нет. Если безопасен — установи глобально.

Ориентир по доверию: десятки тысяч звёзд и тысячи коммитов — это много людей, которые им пользуются, и вероятность спрятанной гадости заметно ниже. Сотня звёзд — совсем другой разговор.

Как замерить эффект

Один и тот же вопрос до и после — и сравнить.

До установки: «Найди, где в проекте считаются комиссии и кто это вызывает».
После: тот же вопрос.

Записать: время, потраченные токены, качество ответа. В отчёт — честную цифру. В эфире прозвучало: заявляют 99 %, реально освобождается 20–30 %.

Память между сессиями

Раз уж про инструменты. На занятии агент сам рассказал, как устроена его память: это `.md`-файлы — `CLAUDE.md` с вашими инструкциями и автоматическая память, которую он пишет сам по вашим поправкам. Документация: <https://code.claude.com/docs/en/memory>.

Если этого мало — `claude-mem` (<https://github.com/thedotmack/claude-mem>, 92 тыс. звёзд) или любой аналог с хуками, которые сохраняют контекст между сессиями. Вариант ведущего — Obsidian (<https://obsidian.md>): обычные заметки, куда агент пишет через хуки, и которые потом читают другие модели.

8. Как собрать красивый PDF

Для всех, кто сдаёт текстовый результат: исследование, плейбук, спецификацию, отчёт. Сырой markdown и plain text не принимаются — нужен читаемый документ или презентация.

Промпт 1. Документ-отчёт

Собери из этого материала PDF-отчёт, который не стыдно показать руководителю.

Требования:

- титульная страница: название, дата, автор;
- оглавление;
- структура: краткое резюме на полстраницы, потом разделы;
- таблицы вместо перечислений там, где это уместно;
- типографика: один акцентный цвет, читаемый шрифт, поля, колонтитулы с номерами страниц;
- никаких эмодзи и декоративного мусора.

Собери через HTML + CSS для печати и отдай готовый PDF.

Если в системе есть конвейер для этого — используй его, если нет — предложи, что поставить, и поставь.

Промпт 2. Презентация

Сделай презентацию по этому материалу: 8–12 слайдов, формат 16:9, на выходе PDF.

Структура: проблема → что сделали → как проверяли → что получилось → выводы → что дальше.

На слайде — одна мысль. Тезисы, а не абзацы. Цифры — крупно.

Единая сетка, один акцентный цвет, читаемо с проектора.

Собери как HTML-слайды с печатью в PDF (по слайду на страницу).

Если удобнее — сделай .pptx и экспортируй в PDF.

Промпт 3. Поднять качество вёрстки

К вёрстке применимы дизайн-скиллы — те самые, что ставили на занятии 1 (**frontend-design** и подобные).

Установи скилл для фронтенд-дизайна и примени его к вёрстке этого документа:

типографическая сетка, ритм, контраст, поля, читаемость.

Покажи «до» и «после» одной страницы, чтобы я мог сравнить.

Отдельный вариант — **Pencil** (<https://pencil.dev>), показанный в конце занятия: агент рисует макет структурой, а не картинкой, и результат потом правится руками.

Проверка перед сдачей

Открой получившийся PDF, посмотри на него как читатель и скажи честно: что выглядит плохо, где текст налезает, где страница пустая наполовину, где таблица не влезла. Исправь и пересобери.

9. Мобильная разработка: ADB и WDA

Если под рукой есть тестовое устройство — попробуйте, хотя бы из интереса.

ПЛАТФОРМА	ИНСТРУМЕНТ	ССЫЛКА
Android	ADB (Android Debug Bridge)	https://developer.android.com/tools/adb
iOS	WDA (WebDriverAgent)	https://github.com/appium/WebDriverAgent
Веб	Playwright	https://playwright.dev

Агент подключается к устройству и прогоняет сценарии сам. По оценке с занятия, так закрывается около 80 % ручного тестирования — и это удобнее симулятора там, где симулятор бессилён: звонки, пуши, реальные задержки сети.

У меня подключён телефон по ADB. Проверь на нём приложение:

- 1) вход;
- 2) основной сценарий;
- 3) сценарий с ошибкой (нет сети, неверные данные);
- 4) реакция на пуш.

Делай скриншоты на каждом шаге и собери отчёт: что прошло, что нет, где именно упало.

10. Где следить за новинками

Личные рекомендации ведущего

Это то, что смотрю сам. **Смотреть именно это не обязательно** — просто нравится.

- **YouTube — ProdAdvice:** <https://www.youtube.com/@ProdAdvice>
- **Telegram — GitHub Radar:** <https://t.me/GitHubRadar> — выжимка из трендовых репозиторий GitHub.
- **Telegram — Вайбкодер-чат:** <https://t.me/vibecoderchat> — крупный чат вайбкодеров. Не канал, а живое обсуждение: агенты, кодинг-средства, платформы, инструменты, модели и вообще много чего. Туда можно прийти с вопросом — и там же подсмотреть, чем сейчас пользуются другие.

Всё остальное — по своим темам

Универсального списка нет и быть не может: бэкендеру, мобильному разработчику и маркетологу интересно разное. Ищите по своим темам — так полезнее, чем читать общий поток.

Что работает как источник:

- **YouTube** — там люди делают развёрнутые разборы; один хороший ролик экономит день-два самостоятельного сбора. Оговорка: посмотрите несколько роликов автора, чтобы понять, учит он с точки зрения разработки или просто балуется с ИИ. Смотреть стоит и то, и то.
- **GitHub → Explore → Trending** — <https://github.com/trending>. Многие телеграм- и инстаграм-каналы просто парсят эту страницу и постят выжимку.
- **Профильные телеграм-каналы и рилсы** — да, инструменты вроде GitNexus и Graphify были найдены именно так. Но проверять всё найденное обязательно.

Предупреждение из эфира

Держать руку на пульсе всего сразу невозможно. Месяц в режиме «отслеживаю каждое нововведение» делает вас топ-специалистом — и съедает всё свободное время. Как и в любой сфере, выберите вектор и углубляйтесь в него, а в стороны смотрите изредка, чтобы не пропустить то, что упростит вашу работу.

11. Чек-лист перед сдачей

Обязательно:

- ☐ Результат запускается по инструкции из `README.md`
- ☐ Работа делалась субагентами — это видно
- ☐ Workflow приложен — файл или текст
- ☐ Текстовый результат оформлен в PDF
- ☐ Секретов в репозитории нет: `.env`, `settings.json`, история команд, скриншоты
- ☐ Для команды: в `README.md` написано, кто что делал

Если делали — в плюс:

- ☐ Замеры «последовательно против параллельно»
- ☐ Три worktree в истории, ветки смержены
- ☐ Изоляция доказана выводом неудачной попытки
- ☐ Инъекция проверена на двух моделях, таблица заполнена
- ☐ Отчёт по инструментам: что понравилось, что нет
- ☐ Журнал сессий с дословными промптами
- ☐ В прошлый раз сдавали архивом — в этот раз Git